Project 3 : " Customer Segmentation Using K-Means "

Installing neccesary libraries

In [1]:
```
%pip install pandas
%pip install numpy
%pip install matplot
%pip install scikit-learn
%pip install seaborn
```

```
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.1-py3-none-any.whl.metadata (5.6 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.7.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_6
4.whl (12.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.9/12.9 MB 5.9 MB/s eta 0:00:00a 0:00:
01
Downloading joblib-1.5.1-py3-none-any.whl (307 kB)
Downloading scipy-1.15.3-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(37.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.7/37.7 MB 6.6 MB/s eta 0:00:0000:010
0:01
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, scipy, joblib, scikit-learn
Successfully installed joblib-1.5.1 scikit-learn-1.7.0 scipy-1.15.3 threadpoolctl-3.
6.0
Note: you may need to restart the kernel to use updated packages.
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /srv/conda/envs/notebook/lib/
python3.10/site-packages (from seaborn) (2.2.6)
Requirement already satisfied: pandas>=1.2 in /srv/conda/envs/notebook/lib/python3.1
0/site-packages (from seaborn) (2.3.0)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /srv/conda/envs/notebook/l
ib/python3.10/site-packages (from seaborn) (3.10.3)
Requirement already satisfied: contourpy>=1.0.1 in /srv/conda/envs/notebook/lib/pyth
on3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /srv/conda/envs/notebook/lib/python3.
10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /srv/conda/envs/notebook/lib/pyt
hon3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.58.4)
Requirement already satisfied: kiwisolver>=1.3.1 in /srv/conda/envs/notebook/lib/pyt
hon3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /srv/conda/envs/notebook/lib/pytho
n3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in /srv/conda/envs/notebook/lib/python3.10/
site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.2.1)
Requirement already satisfied: pyparsing>=2.3.1 in /srv/conda/envs/notebook/lib/pyth
on3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /srv/conda/envs/notebook/lib/
python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /srv/conda/envs/notebook/lib/python3.
10/site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /srv/conda/envs/notebook/lib/python
3.10/site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in /srv/conda/envs/notebook/lib/python3.10/s
ite-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
Note: you may need to restart the kernel to use updated packages.
```

Importing required libraries and Data

In [2]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.neighbors import KNeighborsClassifier

import os

# Load dataset or create synthetic data if file does not exist
if not os.path.exists('customer_data.csv'):
    np.random.seed(42)
    n_samples = 200
    df = pd.DataFrame({  # for emergency when dataset is not available
        'age': np.random.randint(18, 70, size=n_samples),
        'income': np.random.randint(20000, 120000, size=n_samples),
        'frequency': np.random.randint(1, 30, size=n_samples),
        'spending': np.random.randint(100, 10000, size=n_samples)
    })
    df.to_csv('customer_data.csv', index=False)
else:
    df = pd.read_csv('customer_data.csv')
data = pd.read_csv('customer_data.csv')
print(data)
```

```
     customer_id          name  age  income  frequency  spending
0           1001     customer1   48   35718         13     95553
1           1002     customer2   47   44271         16      7687
2           1003     customer3   45   38485         15     75898
3           1004     customer4   59   72271          5     83550
4           1005     customer5   26   34546          9     77299
..           ...           ...  ...     ...        ...       ...
995         1996   customer996   26   73648         10     52166
996         1997   customer997   37   55960         18     99083
997         1998   customer998   30   72681          7     86500
998         1999   customer999   40   79103          7     16595
999         2000  customer1000   54   87506         10     26375

[1000 rows x 6 columns]
```

Preparing Data for our model in required format

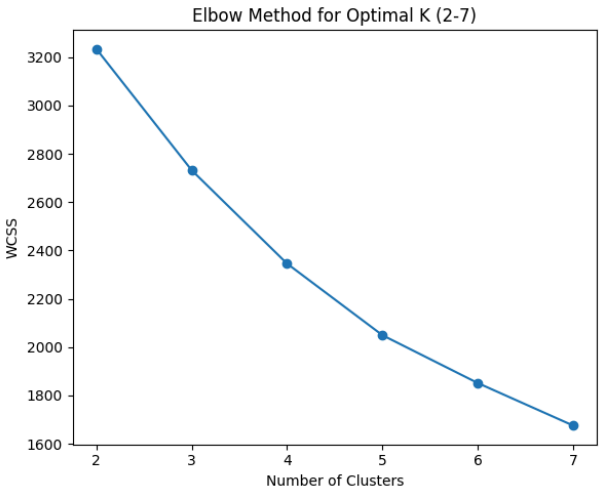
In [3]:
```
# Cleaning column names and ensure all are lowercase, no spaces
df.columns = [col.strip().lower().replace(" ", "_") for col in df.columns]

# Checking for missing values and fill if necessary
if df.isnull().sum().sum() > 0:
    df = df.fillna(df.median(numeric_only=True))

# Selecting relevant features
feature_cols = ['age', 'income', 'frequency', 'spending']
df = df[feature_cols]
print(f"Prepared DataSet\n{df}")
```

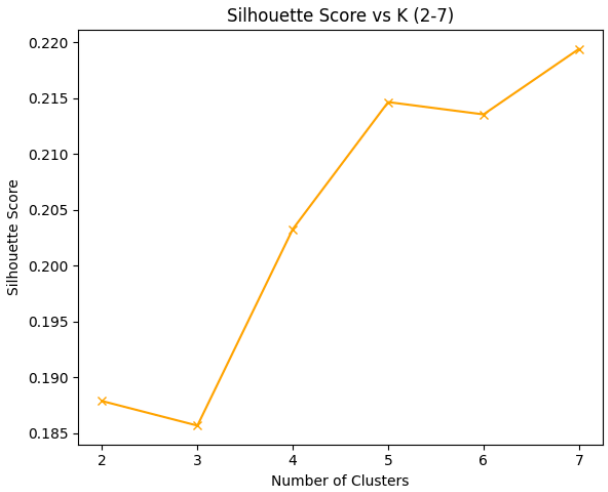
```
features = df[feature_cols]

# Scaling data
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

```
Prepared DataSet
     age  income  frequency  spending
0     48   35718         13     95553
1     47   44271         16      7687
2     45   38485         15     75898
3     59   72271          5     83550
4     26   34546          9     77299
..   ...     ...        ...       ...
995   26   73648         10     52166
996   37   55960         18     99083
997   30   72681          7     86500
998   40   79103          7     16595
999   54   87506         10     26375

[1000 rows x 4 columns]
```

Finding the optimal number of clusters (Elbow Method + Silhouette)

In [4]:
```
wcss = []
silhouette_scores = []
K_range = range(2, 8)
for i in K_range:
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(scaled_features)
    wcss.append(kmeans.inertia_)
    score = silhouette_score(scaled_features, kmeans.labels_)
    silhouette_scores.append(score)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(list(K_range), wcss, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal K (2-7)')

plt.subplot(1, 2, 2)
plt.plot(list(K_range), silhouette_scores, marker='x', color='orange')
plt.xlabel('Number of Clusters')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Score vs K (2-7)')
plt.tight_layout()
plt.show()
```





Selecting 3-5 Meaningful Clusters based on silhouette score

In [5]:
```
optimal_k = K_range[np.argmax(silhouette_scores)]
print(f"Optimal number of clusters (by silhouette score): {optimal_k}")
```

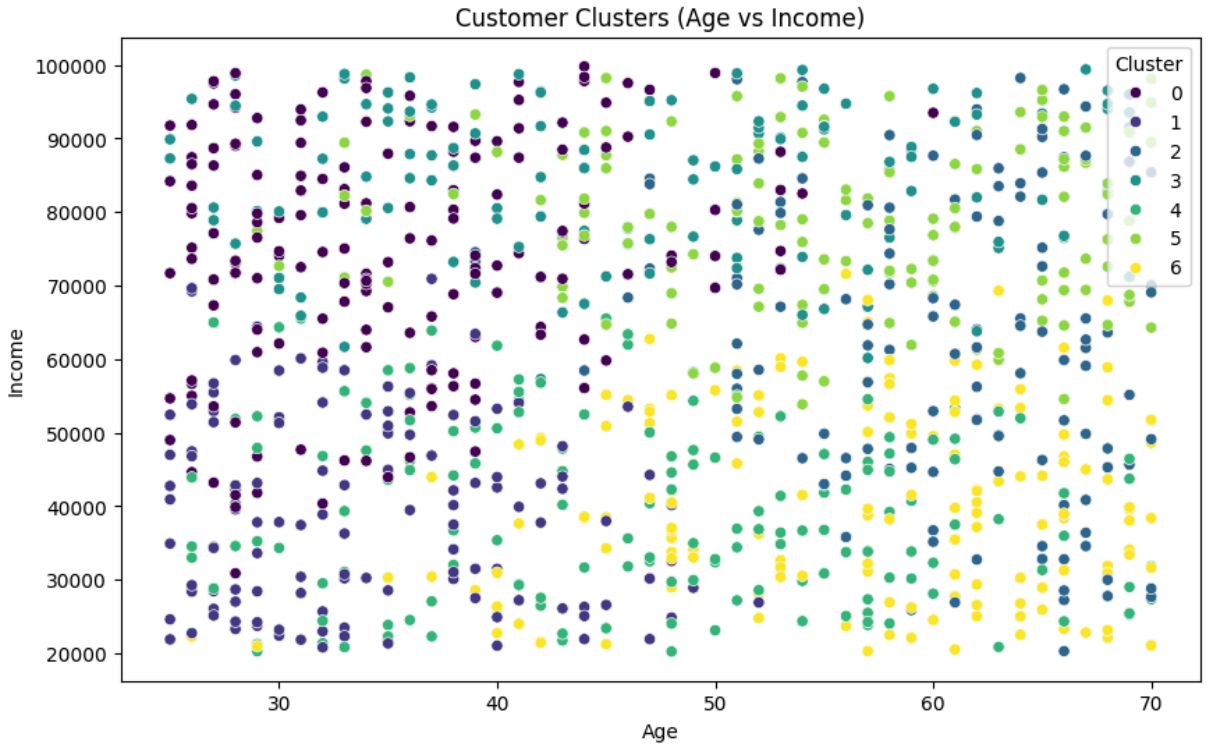
```
Optimal number of clusters (by silhouette score): 7
```

***KMeans with optimal_k and assign cluster label**

In [6]:
```
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(scaled_features)
```

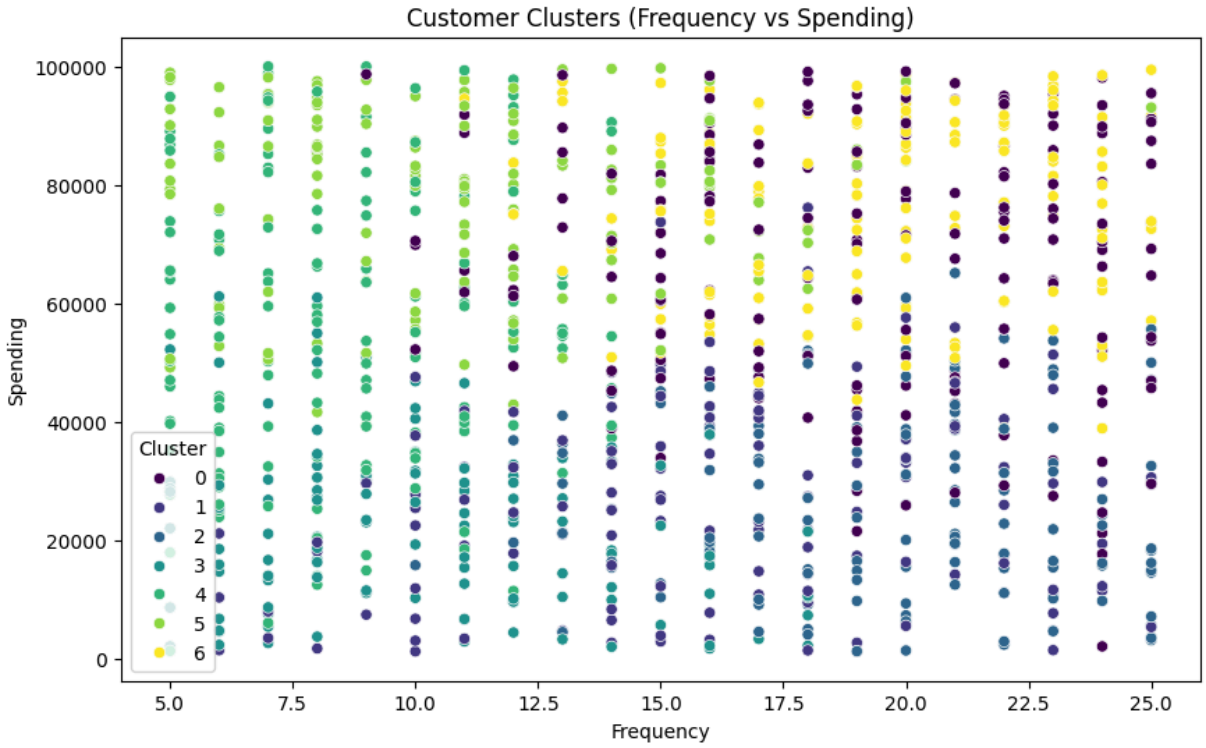
2D Scatter Plot: Age vs Income

In [7]:
```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['age'], y=df['income'], hue=df['cluster'], palette='viridis')
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('Customer Clusters (Age vs Income)')
plt.legend(title='Cluster')
plt.show()
```



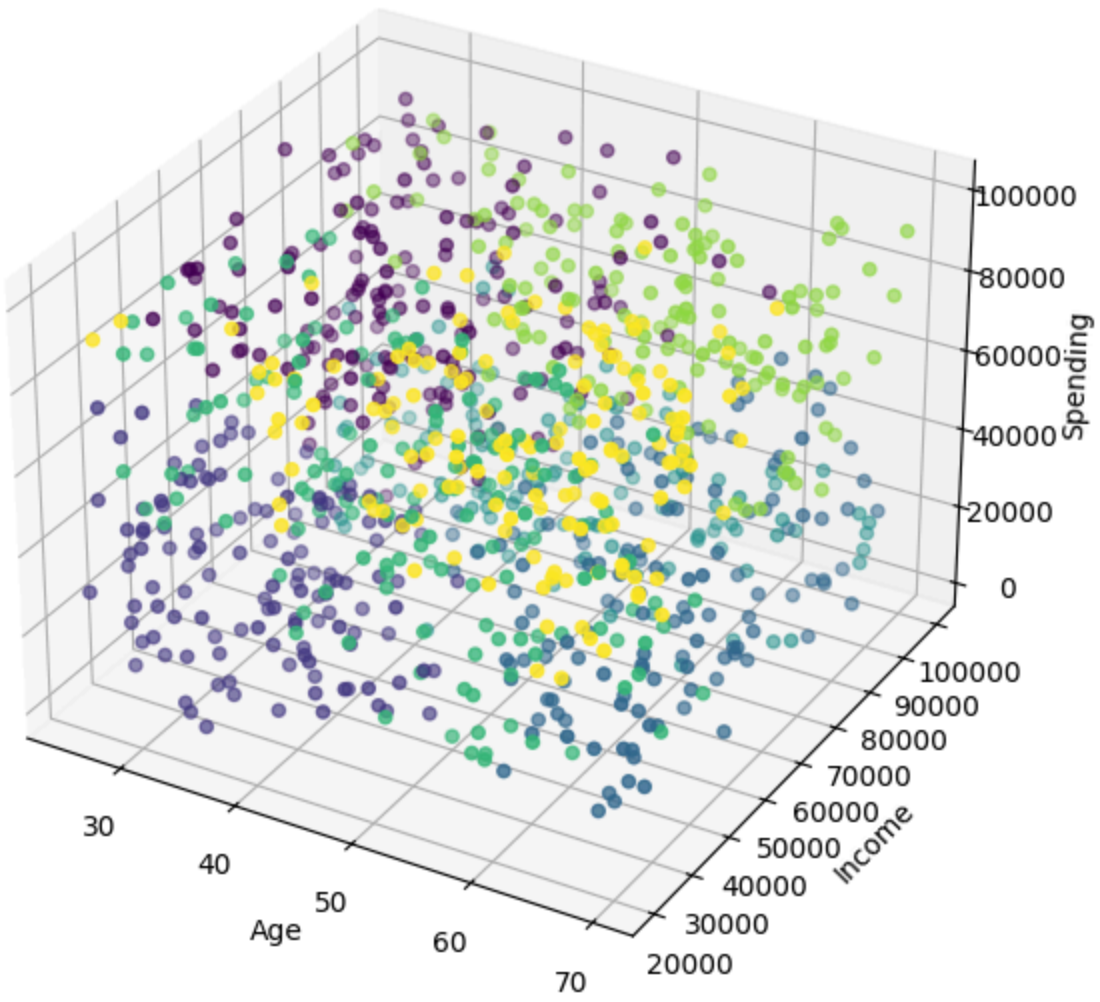
2D Scatter Plot: Frequency vs Spending

In [8]:
```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['frequency'], y=df['spending'], hue=df['cluster'], palette='vi
plt.xlabel('Frequency')
plt.ylabel('Spending')
plt.title('Customer Clusters (Frequency vs Spending)')
plt.legend(title='Cluster')
plt.show()
```



3D Scatter Plot: Age, Income, Spending

In [9]:
```
fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(df['age'], df['income'], df['spending'], c=df['cluster'], cmap='viridis'
ax.set_xlabel('Age')
ax.set_ylabel('Income')
ax.set_zlabel('Spending')
ax.set_title('3D Customer Segmentation')
plt.show()
```



Cluster Summary for Marketing Insights

In [10]:
```
print("\nCluster Summary (mean values):")
summary = df.groupby('cluster')[feature_cols].mean().round(2)
print(summary)
```

```
Cluster Summary (mean values):
           age    income  frequency  spending
cluster
0        35.39  74829.63      19.17  66466.91
1        34.25  39224.45      16.41  25767.59
2        60.58  61971.41      19.61  26523.69
3        45.81  84352.85      10.23  22159.08
4        47.18  39569.92       8.66  56504.62
5        55.02  78245.66      10.78  77865.86
6        55.89  40168.29      19.36  75808.20
```

Cluster sizes

In [11]:
```
print("\nCluster Sizes:")
print(df['cluster'].value_counts().sort_index())
```

```
Cluster Sizes:
cluster
0    170
1    130
2    136
3    118
4    155
5    144
6    147
Name: count, dtype: int64
```

5 Meaningful Customer types for Marketing

In [12]:
```
print("\nCustomer Segment Profiles:")
for cluster_id, row in summary.iterrows():
    desc = []
    if row['income'] > summary['income'].mean() and row['spending'] > summary['spen
        desc.append("High-income, high-spending (Premium Customers)")
    elif row['age'] < summary['age'].mean() and row['frequency'] > summary['frequen
        desc.append("Young, frequent shoppers (Enthusiasts)")
    elif row['spending'] < summary['spending'].mean() and row['frequency'] < summar
        desc.append("Low-spending, infrequent (Budget/Occasional Shoppers)")
    elif row['age'] > summary['age'].mean():
        desc.append("Older customers (Mature Segment)")
    else:
        desc.append("Average/Regular Customers")
    print(f"Cluster {cluster_id}: {', '.join(desc)}")
    print(row)
    print("-" * 40)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(scaled_features, df['cluster'])
```

```
Customer Segment Profiles:
Cluster 0: High-income, high-spending (Premium Customers)
age             35.39
income       74829.63
frequency       19.17
spending     66466.91
Name: 0, dtype: float64
----------------------------------------
Cluster 1: Young, frequent shoppers (Enthusiasts)
age             34.25
income       39224.45
frequency       16.41
spending     25767.59
Name: 1, dtype: float64
----------------------------------------
Cluster 2: Older customers (Mature Segment)
age             60.58
income       61971.41
frequency       19.61
spending     26523.69
Name: 2, dtype: float64
----------------------------------------
Cluster 3: Low-spending, infrequent (Budget/Occasional Shoppers)
age             45.81
income       84352.85
frequency       10.23
spending     22159.08
Name: 3, dtype: float64
----------------------------------------
Cluster 4: Average/Regular Customers
age             47.18
income       39569.92
frequency        8.66
spending     56504.62
Name: 4, dtype: float64
----------------------------------------
Cluster 5: High-income, high-spending (Premium Customers)
age             55.02
income       78245.66
frequency       10.78
spending     77865.86
Name: 5, dtype: float64
----------------------------------------
Cluster 6: Older customers (Mature Segment)
age             55.89
income       40168.29
frequency       19.36
spending     75808.20
Name: 6, dtype: float64
----------------------------------------
```

Out[12]: